

Tsuru — Documentação Técnica

Portal de Gestão de PD&I e Lei do Bem da Bellube Versão deste documento: 03/07/2026 · Autor original: Daniel Mendes Teixeira de Souza · Manutenção: Rodrigo de Souza Faria (T&I)

1. Resumo executivo

O Tsuru é um sistema web interno que substitui o controle do ciclo de Pesquisa, Desenvolvimento e Inovação (PD&I) da Bellube — hoje feito em planilhas soltas, e-mail e memória institucional — por um portal auditável, com histórico completo de decisões e integração direta com o Sankhya ERP e o Microsoft 365.

Ele resolve três problemas concretos que a empresa tinha antes:

1. **Não havia rastreabilidade** de quem decidiu o quê, quando, e por quê, ao longo do funil de aprovação de um projeto de inovação.
2. **O dossiê de defesa da Lei do Bem era remontado do zero todo ano**, com risco real de glosa por "cópia sem evidência de progresso técnico distinto" — um dos critérios mais comuns de autuação do Comitê de Apoio Técnico (CAT) do MCTI.
3. **Não havia visão executiva** de quantos projetos estão de fato em andamento versus parados esperando alguém — a Diretoria só sabia o status de um projeto perguntando diretamente para quem estava tocando.

O Tsuru está em produção desde maio/2026, com **119 usuários reais** (colaboradores, gestores, diretoria) e **mais de 30 projetos/sugestões PD&I** cadastrados, cobrindo desde a submissão inicial de uma ideia até a composição final do dossiê de defesa perante o MCTI.

2. O problema que motivou o projeto

Antes do Tsuru, o ciclo de PD&I da Bellube funcionava assim:

- Um colaborador tinha uma ideia de melhoria e comunicava por e-mail ou verbalmente ao supervisor.
- A aprovação de elegibilidade (Lei do Bem) acontecia numa reunião ou troca de e-mails, sem registro formal do critério aplicado.
- No fim do ano-base, alguém sentava e tentava reconstruir, de memória e de e-mails antigos, o que tinha acontecido em cada projeto para montar o dossiê de defesa.
- Projetos que atravessavam mais de um ano-base corriam risco real de ter o relato copiado do ano anterior — motivo comum de glosa (recusa do benefício fiscal) pelo CAT/MCTI.
- Não havia como saber, num relance, quantos projetos estavam realmente "andando" e quantos estavam simplesmente esquecidos esperando alguém responder.

Esse cenário gera dois riscos concretos para a empresa: **risco fiscal** (perda do benefício da Lei do Bem por documentação fraca) e **risco de gestão** (projetos de inovação que morrem silenciosamente por

falta de acompanhamento).

3. O que o Tsuru faz

3.1. Funil INOVA BEL (6 etapas, 17 estados, 20 transições possíveis)

Toda ideia de melhoria passa por uma esteira com passagens formais e auditáveis:

1. Colaborador registra a sugestão → 2. Supervisor da área aprova
→ 3. Time de T&I faz a triagem técnica (N1 + N2)
→ 4. Diretoria valida → 5. FI Group (consultoria externa) dá o parecer de elegibilidade
→ 6. Vira Projeto de Fato (INOVA BEL oficial)

Cada passagem entre etapas fica registrada permanentemente — **quem** aprovou, **quando**, e **por quê** (quando exigido). Esse registro (`DemandTransition`) é somente-inserção: nem um bug de código consegue reescrever o histórico, porque a aplicação bloqueia qualquer alteração ou exclusão desses registros (só é permitido criar novos).

O funil também aceita desvios do caminho feliz a qualquer momento: uma sugestão pode ser devolvida para revisão, cancelada, arquivada, ou convertida diretamente numa tarefa de um projeto já existente (quando a ideia é uma melhoria pontual válida mas não chega a ser elegível para a Lei do Bem).

3.2. Triagem N1 / Avaliação N2 / Composição N3 — nativos da ferramenta

O Tsuru modela as três fases da consultoria de Lei do Bem diretamente no fluxo de trabalho, não como formulário à parte:

- **N1 (triagem binária)** — checklist de 5 perguntas SIM/NÃO (rotina operacional? solução de prateleira? TRL fora da janela elegível? etc.). Uma única resposta SIM já reprova o projeto automaticamente, evitando que se perca tempo redigindo um dossiê para algo inelegível.
- **N2 (avaliação discursiva)** — captura motivação, benchmark anterior, barreira técnica, metodologia experimental, stack tecnológico e resultado obtido. Esses campos são os que efetivamente sustentam a defesa técnica perante o CAT.
- **N3 (composição final de defesa)** — consolidação anual com critérios de sucesso mensuráveis, benefícios operacionais, barreiras resolvidas e não resolvidas, e geração de PDF do dossiê pronto para arquivamento/contestação.

3.3. Painel de Atualizações — visão executiva em tempo real

Tela dedicada (`/atualizacoes`) que separa, de forma objetiva:

-  **Em andamento** — projetos onde alguém está de fato trabalhando agora.
-  **Em standby** — projetos parados esperando uma decisão de terceiro (autor precisa responder, Diretoria precisa decidir, ou a FI Group precisa dar parecer).
-  **Concluídas recentemente** — atalho para o que fechou (com qualquer desfecho).
- **Feed de atividade** — linha do tempo única mesclando transições de estado, comentários e movimentações de tarefas em todo o portfólio, para quem quer entender "o que aconteceu essa

semana" sem entrar projeto por projeto.

3.4. Kanban interno por projeto

Cada projeto tem seu próprio quadro de tarefas (backlog → a fazer → em andamento → em revisão → concluída), com prioridade, responsável, prazo, dependências entre tarefas, checklists, timer de horas trabalhadas e comentários.

3.5. Organograma como árvore genealógica real

A hierarquia da empresa é modelada literalmente (quem reporta para quem, via `supervisor_id`), não como uma lista plana agrupada por área. A Diretoria aparece no topo; abaixo dela, cada liderança e sua cadeia de subordinados, recursivamente — igual a uma árvore genealógica de fato.

3.6. Integração com Sankhya ERP

- Sincronização de colaboradores, parceiros PJ e projetos via gateway REST genérico do Sankhya (`CRUDServiceProvider`), com cache local configurável.
- Reconciliação de identidade entre Sankhya e Microsoft Entra ID: o Sankhya usa e-mails de "rota" que são reatribuídos entre pessoas ao longo do tempo (ex.: a mesma caixa `vendas44@` teve 4 titulares diferentes em 3 anos) — o Tsuru usa o Entra ID como fonte de verdade da identidade atual e o Sankhya como fonte de cargo/área/atividade.
- **Correção importante feita nesta fase:** a primeira rotina de inativação automática de usuários usava apenas "ausência de login no Sankhya por 90+ dias" como critério — e errou em 16 de 22 casos, marcando como inativos colaboradores que simplesmente não usam aquele sistema no dia a dia mas seguem empregados. A correção cruzou contra a tabela de RH do próprio Sankhya (`TFPFUN`, campo de data de demissão real) e restaurou os 16 contratos ainda ativos, mantendo apenas as 6 inativações com demissão de fato confirmada. Essa é uma lição de engenharia relevante: **nunca inferir status de emprego a partir de atividade de login em um sistema periférico.**

3.7. Automações compatíveis com Microsoft Power Automate

Webhooks de saída (disparados por eventos do funil, como "demanda submetida" ou "projeto aprovado") e API de entrada REST autenticada por token pessoal, permitindo que fluxos no Power Automate criem tarefas ou comentários no Tsuru automaticamente.

3.8. Inteligência artificial aplicada

- Resumo executivo de projeto sob demanda (estado atual, riscos, próximos passos, aderência à Lei do Bem).
- Insight agregado de portfólio (visão do conjunto de projetos).
- Suporte a múltiplos provedores de LLM (OpenAI, Anthropic, Gemini ou modelo local), configurável por administrador.

3.9. API administrativa completa + servidor MCP para agentes de IA

Construído nesta fase: uma API REST administrativa (`Api::V1::Admin::*`) que expõe o Tsuru inteiro — usuários, projetos, tarefas, áreas, organograma, relatórios — para integração externa autenticada por token de administrador.

Sobre essa API foi construído o **tsuru-mcp**: um servidor no padrão MCP (Model Context Protocol, o mesmo usado por assistentes de IA como o Claude) com 20 ferramentas, que permite que um agente de código (Claude Code ou qualquer outro agente compatível) administre o Tsuru remotamente — criar e mover projetos pela esteira, gerenciar usuários, consultar o organograma, gerar relatórios de IA — tudo sem precisar abrir o navegador. Repositório próprio, testado ponta a ponta contra o ambiente de produção.

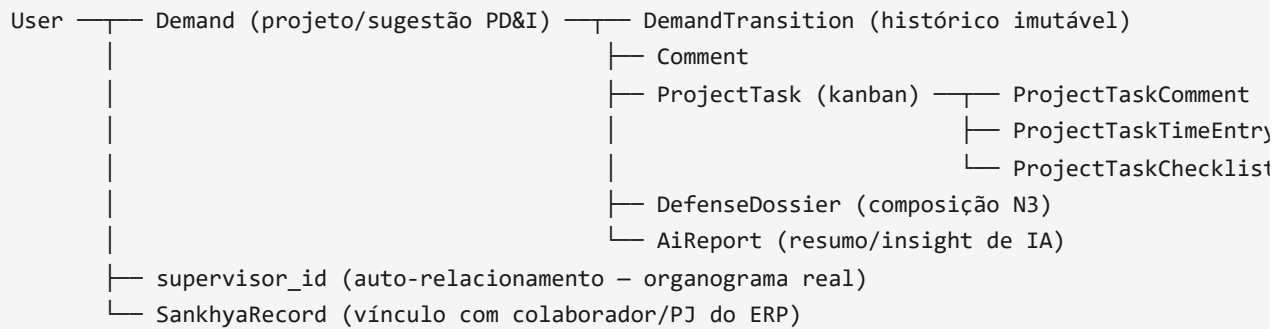
3.10. Exportação e relatórios

Exportação em CSV/XLSX de demandas, tarefas e timesheet (incluindo os códigos de vínculo com o Sankhya, quando configurados), disponível para todos os perfis de gestão, não só administradores.

4. Arquitetura e stack tecnológico

Camada	Tecnologia
Backend	Ruby on Rails 8.1
Frontend	Hotwire (Turbo + Stimulus), Tailwind CSS v4 — sem framework JS pesado (SPA)
Banco de dados	PostgreSQL 16
Autenticação	Devise, com autenticação de dois fatores (2FA)
Autorização	Pundit (controle de acesso por papel: colaborador, gestor, analista PD&I, admin, diretoria, consultoria FI)
Auditoria	PaperTrail (histórico de alterações em todos os registros) + DemandTransition (log imutável, somente-inserção, de toda mudança de estado)
Máquina de estados	<code>state_machines-activerecord</code>
Geração de PDF	Prawn
Paginação	Pagy
Integração externa	Faraday (cliente HTTP para o gateway REST do Sankhya), OAuth2
IA generativa	Clientes para OpenAI, Anthropic e Gemini, mais suporte a modelo local
Testes automatizados	RSpec (specs de model e de request), FactoryBot, Playwright (testes end-to-end reais contra produção)
Servidor MCP	Node.js 20+ / TypeScript, <code>@modelcontextprotocol/sdk</code>

4.1. Modelo de dados (visão simplificada)



4.2. Segurança

- Autenticação com 2FA disponível para todos os usuários.
- Controle de acesso por papel em toda a aplicação (Pundit), incluindo a API administrativa.
- Histórico de decisões protegido a nível de aplicação contra alteração/exclusão retroativa (`readonly?` sobrescrito em `DemandTransition`).
- Token de API pessoal (para integrações externas) gerável e renovável pelo próprio usuário.
- Comunicação atual em HTTP simples (sem TLS) dentro da rede interna/VPN — recomendação de colocar HTTPS na frente antes de qualquer exposição externa do serviço.

5. Estado atual (03/07/2026)

- **119 usuários ativos**, reconciliados entre Sankhya e Microsoft 365 — apenas 6 desligamentos confirmados nesta rodada de limpeza (correção de um erro inicial que teria desligado indevidamente 16 colaboradores ainda empregados).
- **Mais de 30 projetos/sugestões** cadastrados, cobrindo desde ideias em rascunho até projetos "de fato" (INOVA BEL oficial) e projetos concluídos.
- **Organograma real** modelado com a hierarquia de Diretoria, gestores intermediários e colaboradores.
- **API administrativa e servidor MCP** publicados e testados contra produção, prontos para uso por agentes de IA e integrações externas.
- **Painel de Atualizações** em produção, dando visão executiva imediata de quantos projetos estão de fato avançando versus parados.

6. Débitos técnicos conhecidos (transparência)

- Jobs em segundo plano (processamento assíncrono, ex.: recorrência de tarefas, e-mails adiados) ainda não têm um processo dedicado rodando em produção — código pronto, falta infraestrutura de execução.
- Comunicação HTTP sem TLS dentro da rede interna — adequado para uso via VPN, mas deve ganhar HTTPS antes de qualquer exposição maior.
- Cerca de 55 usuários importados do Microsoft 365 ainda não têm área/departamento atribuído no Tsuru (o Sankhya não tinha grupo correspondente para essas contas — em sua maioria de empresas

ligadas, como Sion Lubrificantes e Corporate Imóveis).

7. Repositórios

- **Portal:** github.com/alucardigo/tsuru-portal (privado, espelhado em `bellube/tsuru-portal`) — código-fonte completo da aplicação Rails. 86 commits, de 19/05/2026 a 03/07/2026.
- **Servidor MCP:** github.com/alucardigo/tsuru-mcp (privado, espelhado em `bellube/tsuru-mcp`) — servidor de integração para agentes de IA.

Ambos versionados em duas contas GitHub (pessoal e corporativa Bellube), mantidos sincronizados.